1. Que contient une classe en C++?

[ ] Données **[ ]** Fonctions **[ ]** Les données et les fonctions **[ ]** Des tableaux

2. Combien de spécificateurs d'accès existe ?

[ ] 1 **[ ]** 2 **[ ]** 3 **[ ]** 4

3. ___ est utilisé pour définir un membre d'une classe extérieurement ?

[ ] # **[ ]** :: **[ ]**!! $ **[ ]** :

4. Quel opérateur, un objet pointeur d'une classe utilise pour accéder à ses membres de données et à ses fonctions membres?

[ ] :: **[ ]** . **[ ]** -> **[ ]**:

5. Quelle est la sortie du code suivant ?

```
#include <iostream>
using namespace std;
class MaClasse
{ int var;
  public: int write(int i) const {
              var = i;
            }
   int read() const { return var; }
};
int main(int argc, char const *argv[])
{ MaClasse obj;
  obj.write(2);
   cout << obj.read();
}
```

[ ] 2 **[ ]** 3 **[ ]** Erreur de compilation **[ ]** Aucune de ces réponses n'est vraie.

6. Lequel est une déclaration de classe valide ?

[ ] public classe A {} **[ ]** classe A {} **[ ]** classe A {int x;}; **[ ]**object A {int x;};

7. Les membres d'une classe en C++ sont par défaut _____

[ ] private **[ ]** protected **[ ]**public **[ ]** public et protected

8. Quelle est la syntaxe correcte pour accéder à un membre statique d'une classe ?

```
class A
{ public:  static int val;
} ;
```

[ ] A->val **[ ]** A.val **[ ]** A::val **[ ]** A^val

9. Un objet const d'une classe peut appeler une fonction membre non-const de la classe.

[ ] Vrai **[ ]** Faux

10. Si un argument d'une fonction est déclaré en tant que const, alors _____.

[ ] La fonction peut modifier l'argument. **[ ]** La fonction ne peut pas modifier l'argument.
[ ] La fonction ne peut pas avoir un argument const.
[ ] Aucune de ces réponses n'est vraie.

10. Quelle est la sortie du code suivant ?

```
#include<iostream>
using namespace std;
int main()
{int a = 1, b = 1, c = 1;
cout << (++a or ++b and ++c ) << endl;
cout << a << " " << b << " " << c ;
return 0;
}
```

[ ] 0 **[ ]** 1 **[ ]** 1 **[ ]** 1
2 1 1 2 1 1 2 2 2 2 2 1

11. Quelle est la sortie du code suivant ?

```
#include<iostream>
using namespace std;
int main()
{int a = 1 , b = 1;
cout << ( ++a and ++b ) << endl;
cout << a << " " << b;
return 0;
}
```

[ ] 1 **[ ]** 0 **[ ]** 1 **[ ]** Erreur de compilation.
1 1 2 2 2 2

12. Les valeurs par défaut d'une fonction sont spécifiées lorsque ____.

[ ] la fonction est définie **[ ]** la fonction est déclarée
[ ] Les deux A et B sont vrais. **[ ]** Aucune de ces réponses n'est vraie.

13. Les fonctions membres d'une classe peuvent-elles être des fonctions amis d'une autre classe ?

[ ] Oui **[ ]** Non

14. Les constructeurs peuvent-ils être surchargés ?

[ ] Oui **[ ]** Non

15. La surcharge de fonction peut également être obtenue si deux fonctions ou plus ne diffèrent que par leur type de retour.

[ ] Oui **[ ]** Non

16. Quel est le rôle principal du constructeur d'une classe ?

[ ] Allouer la mémoire nécessaire à une instance de cette classe
[ ] Initialiser une instance de cette classe
[ ] Renvoyer l'adresse de l'objet **[ ]** Défragmenter la mémoire

17. Quel est le rôle principal du destructeur d'une classe ?

[ ] Libérer la mémoire occupée par un objet **[ ]** Détruire un objet
[ ] Libérer les ressources de l'objet **[ ]** Défragmenter la mémoire

18. Dans quelle situation la liste d'initialisation des membres est-elle indispensable ?

[ ] Pour initialiser des membres de type pointeur
[ ] Pour initialiser les membres 'static'
[ ] Pour initialiser les membres de type référence ou const
[ ] Pour initialiser des membres de type tableau

19. Identifier l'assertion erronée au sujet des fonctions 'inline' :

[ ] Elles peuvent contenir plusieurs instructions
[ ] Elles doivent être 'static'
[ ] Elles améliorent les performances du programme
[ ] Elles sont expansées à l' endroit de chaque appel

20. Quelle syntaxe permet de modifier la cible d'un pointeur p ?

[ ] *p = a; **[ ]** p = a; **[ ]** &p = a; **[ ]** c'est impossible

21. Quelle syntaxe permet de modifier la cible d'une référence r ?

[ ] *r = a; **[ ]**r = a; **[ ]**&r = a; **[ ]** c'est impossible

22. Dans une classe, les constructeurs de ses objets membres sont appelés_____le code de son constructeur.

[ ] avant **[ ]** après **[ ]** cela dépend du compilateur.

23. Dans une classe, les destructeurs de ses objets membres sont appelés _____le code de son destructeur.

[ ] avant **[ ]** après **[ ]** cela dépend du compilateur.

24. Comment créer un objet si tous les constructeurs de sa classe sont privés ?

[ ] C'est impossible **[ ]** Il faut préciser le namespace **[ ]** Il faut passer par une méthode statique
[ ] Seules les classes dérivées peuvent le faire

25. Les constructeurs des classes de base directes d'une classe sont appelés _____ ceux de ses objets membres.

[ ] avant **[ ]** après **[ ]** cela dépend du compilateur.

26. Les destructeurs des classes de base directes d'une classe sont appelés _____ceux de ses objets membres.

[ ] avant **[ ]** après **[ ]** cela dépend du compilateur.

27. Lequel des énoncés suivants est correct ?

[ ] Le mot clé « struct » est requise en C et C++ tout en déclarant un objet de structure.
[ ] Le mot clé « struct » n'est pas requis en C mais requis en C++ lors de la déclaration d'un objet de structure.
[ ] Le mot clé « struct » n'est pas requis en C++ mais requis en C lors de la déclaration d'un objet de structure.
[ ] Le mot clé « struct » n'est pas requise en C et C++ lors de la déclaration d'un objet de structure.

28. Lequel des énoncés suivants est correct ?

[ ] Une structure ne peut pas avoir de fonction membre en C, mais il est possible en C++.
[ ] Une structure ne peut pas avoir de fonction membre en C++ mais il est possible en C.
[ ] Une structure ne peut pas avoir de fonction membre en C et en C++.
[ ] Une structure peut avoir une fonction membre en C et en C++.

29. Lequel des énoncés suivants est correct à propos du pointeur this en C++ ?

[ ] Le pointeur **this** est passé comme argument caché dans toutes les fonctions d'une classe.
[ ] Le pointeur **this** est passé en tant qu'argument caché dans toutes les fonctions non statiques d'une classe.
[ ] Le pointeur **this** est passé en tant qu'argument caché dans toutes les fonctions statiques d'une classe.
[ ] Aucune de ces réponses n'est vraie.

30. Quelle est la sortie du code C++ suivant ?

```
#include <iostream>
using namespace std;
 class A
{          static int val;
  public:void write(int i){ val = i; }
           void read(){  cout << val;  }
};
int main(int argc, char const *argv[])
{  A a = A();
  a.write(10);
  a.read();
  return 0;
}
```

[ ] 10 **[ ]** Segmentation fault **[ ]** Erreur de compilation **[ ]** Aucune de ces réponses n'est vraie

31. Quelle est la sortie du code C++ suivant ?

```
#include <iostream>
using namespace std;
 class A
{  static int val;
   public:   void write(int i){    val = i;  }
             void read(){  cout << val;  }
};
int A::val = 55;
```

```
int main(int argc, char const *argv[])
{  A a = A();
   a.write(10);
   a.read();
   return 0;
}
```

[ ] 55 **[ ]** 10 **[ ]** Erreur de compilation **[ ]** Aucune de ces réponses n'est vraie

32. Quelle est la sortie du code C++ suivant ?

```
#include <iostream>
using namespace std;
 class A
{     int val = 55;
   public:   void write(int i){  val = i;  }
             static void read(){  cout << val;  }
};
```

```
int main(int argc, char const *argv[])
{  A a = A();
   a.write(10);
   a.read();
   return 0;
}
```

[ ] 55 **[ ]** 10 **[ ]** Erreur de compilation **[ ]** Aucune de ces réponses n'est vraie

33. Quelle est la sortie du code C++ suivant ?

```
#include<iostream>
using namespace std;
int a = 55;
void fun()
{   int a = 20;
    {   int a = 10;
        cout << ::a << endl;
    }
}
```

```
int main()
{     fun();
     return 0;
}
```

[ ] 10 **[ ]** 20 **[ ]** 55 **[ ]** Aucune de ces réponses n'est vraie

34. Quelles fonctions de classe s'appellent des fonctions inline?

[ ] Toutes les fonctions déclarées à l'intérieur d'une classe
[ ] Toutes les fonctions définies en dehors de la classe
[ ] Toutes les fonctions définies à l'intérieur ou avec le mot clé inline
[ ] Toutes les fonctions accédant aux membres statiques de la classe

35. Quelle est la sortie du code C++ suivant ?

```
#include<iostream>
using namespace std;
class Point {
private:
    int x, y;
public:
    Point(int a, int b);
};
Point::Point(int a = 0, int b = 0)  {
    x = a;
    y = b;
    cout << "Constructeur appelé";
}
```

```
int main()
{     Point p1, *p2;
      return 0;
}
```

[ ] Affiche « Constructeur appelé » une seule fois.
[ ] Affiche « Constructeur appelé » deux fois.
[ ] N'affiche rien.
[ ] Erreur de compilation.

36. Qui a créé le C++ ?

[ ] Rasmus Lerdorf **[ ]** Bjarne Stroustrup **[ ]** James Gosling **[ ]** Ken Thompson

37. Lequel des types suivants n'est pas présent en C mais présent en C++ ?

[ ] void **[ ]** bool **[ ]** float **[ ]** int

38. Quel est la sortie du code suivant en C++ ?

```
#include <iostream>
using namespace std;
int main()
{   char tab[4] = "abcd";
    cout << tab;
    return 0;
}
```

[ ] abcd **[ ]** Avertissement

[ ] Erreur : la chaîne de caractères dans le tableau est trop longue.

[ ] Aucune de ces réponses n'est vraie.

39. Quelle est la sortie du programme C++ suivant ?

```
#include<iostream>
using namespace std;
class Point {
private:
    int x,  y;
public:
    Point(int a = 0, int b = 0);    // Constructeur Normal
   Point(const Point &p); // Constructeur de Copie
};
Point::Point(int a, int b)  {
    x = a;
    y = b;
    cout << "Constructeur normal est appelé\n";
}
Point::Point(const Point &p) {
   x = p.x;
   y = p.y;
   cout << "Constructeur de copie est appelé\n";
}
int main()
{  Point *p1, *p2;
   p1 = new Point(5, 10);
   p2 = new Point(*p1);
   Point p3 = *p1;
   Point p4;
   p4 = p3;
   return 0;
}
```

[ ] Constructeur normal est appelé
Constructeur de copie est appelé
Constructeur de copie est appelé
Constructeur normal est appelé

[ ] Constructeur normal est appelé
Constructeur normal est appelé
Constructeur normal est appelé
Constructeur de copie est appelé
Constructeur de copie est appelé
Constructeur normal est appelé

[ ] Constructeur de copie est appelé
Constructeur normal est appelé
Constructeur de copie est appelé
Constructeur de copie est appelé
Constructeur normal est appelé

[ ] Aucune de ces réponses n'est vraie.

40. Quelle est la sortie du programme C++ suivant ?

```
#include<iostream>
using namespace std;
  class MaClasse {
            int val;
  public: MaClasse(int v);
};
MaClasse::MaClasse(int v) {
    val = v;
    cout << "Constructeur appelé\n";
}
int main() {
    MaClasse tab[100];
    return 0;
}
```

[ ] Affiche « Constructeur appelé »

[ ] N'affiche rien

[ ] Erreur de compilation

[ ] Aucune de ces réponses n'est vraie.

41. Lesquelles des instructions suivantes sont valables pour allouer de la mémoire dynamiquement pour un entier en C++ ?

[ ] int *ptr = new int(100);
[ ] int *ptr; ptr=new int; *ptr=100;
[ ] int *ptr=NULL; ptr=new int; *ptr=100;
[ ] Toutes les réponses sont vraies

42. La signature d'une fonction c'est :

[ ] Son nom et le type de sa valeur de retour
[ ] Son nom et les noms de ses paramètres
[ ] Son nom et les types de ses paramètres
[ ] Son nom, les types de ses paramètres et le type de sa valeur de retour

43. Si une classe B hérite d'une classe A, on dit que :

a) B spécialise A **[ ]** vrai **[ ]** faux
b) B généralise A **[ ]** vrai **[ ]** faux
c) B possède au moins tous les champs et les méthodes de A **[ ]** vrai **[ ]** faux
d) A possède au moins tous les champs et les méthodes de B **[ ]** vrai **[ ]** faux
e) Toute instance de B peut être considérée comme un A **[ ]** vrai **[ ]** faux
f) Toute instance de A peut être considérée comme un B **[ ]** vrai **[ ]** faux

44. Si les classes Pomme **et** Orange **dérivent de la classe** Fruit **et la classe** Golden **dérive de la classe** Pomme **alors on peut écrire :**

a) Fruit * tab = new Orange [10] ; **[ ]** vrai **[ ]** faux
b) Fruit *tab = new Fruit [10] ; **[ ]** vrai **[ ]** faux
c) Golden *tab = new Pomme [10] ; **[ ]** vrai **[ ]** faux
d) Golden *tab = new Orange [10] ; **[ ]** vrai **[ ]** faux

- Parmi les quatre propositions ci-dessus, laquelle permet de créer un tableau pouvant contenir des oranges, des pommes et des golden (a, b, c ou d)? Inscrire le numéro de la réponse [ ]

45. En C++, la déclaration d'une classe :

[ ] ne doit contenir qu'1 seul constructeur **[ ]** doit toujours contenir 2 constructeurs
[ ] doit contenir au moins 1 constructeur **[ ]** peut contenir 0 ou plusieurs constructeurs

46. En C++, dans l'instruction : C c1; C c2=c1; **la déclaration de** c2 **fait intervenir :**

[ ] la surcharge de l'opérateur = de la classe C **[ ]** le destructeur la classe C
[ ] le constructeur de copie de la classe C **[ ]** rien du tout

47. Après l'exécution du code suivant :

```
std::vector<int> v = { 1, 3, 6 };
v.push_back(9);
v.insert(v.begin()+2,-1);
```

v **est égal à :**

[ ] 1 3 -1 -1 9 **[ ]** 1 3 -1 6 9 **[ ]** 1 -1 3 6 9 **[ ]** 9 1 3 -1 6

47. Observer le fragment de programme suivant :

```
1  vector<int> lireTableau(int n) {
2      vector<int> t;
3      t = vector<int>(n);
4      for ( int i=0; i < n; i++ ) {
5          cin >> t[i];
6      }
7      return t;
8  }
9
10 int main() {
11     vector<int> t = lireTableau(10);
12     for ( int i = 0; i < t.size(); i++ ) {
13         cout << t[i] << endl;
14     }
15 }
```

a) Quel est le type de retour de la fonction lireTableau ?

b) Indiquer le numéro de la ou les lignes de la fonction lireTableau qui

Déclarent le tableau t :

Allouent le tableau t :

Initialisent le tableau t :

c) En une phrase, décrire ce que fait ce programme.

48. (1) Écrire une fonction nommée contientNegatif **qui prend en paramètre un tableau d'entiers** t **et renvoie** true **si** t **contient au moins un nombre strictement négatif et** false **sinon.**

```
bool contientNegatif(vector<int> t) {




}
```

(2) Si ce n'est déjà fait, réécrivez la fonction précédente en utilisant une boucle for each (parcours par les éléments) **:**

```
bool contientNegatif(vector<int> t) {




}
```

49. Choisir la bonne déclaration à propos des objets string **en C++ ?**

[ ] Les objets string doivent être terminés par le caractère nul (‘\0’)
[ ] Les objets string ont une taille statique
[ ] Les objets string ont une taille dynamique
[ ] Les objets string utilisent davantage de mémoire que nécessaire.

50. Quelle fonction est utilisée pour obtenir la longueur d’une chaîne de caractères en C++?

[ ] str.length()
[ ] str.size()
[ ] str.max_size()
[ ] Les deux fonctions size() et length()

51. Quel fichier d’en-tête est utilisé pour inclure les fonctions de string dans C++?

[ ] #include <string.cpp>
[ ] #include <string.h>
[ ] #include <string>
[ ] #include <cstring>

52. Choisir la bonne option :

string* a, b;

[ ] a est un pointeur sur une chaîne de caractères, b est une chaîne de caractères.
[ ] b est un pointeur sur une chaîne de caractères, a est une chaîne de caractères
[ ] a et b sont des pointeurs de type String.
[ ] Aucune de ces réponses n’est vraie.

5.3 A quoi correspond le mot clé virtual ?

[ ] Il permet d'invoquer une méthode codée en Java qui s'exécute dans une JVM (Java Virtual Machine).
[ ] Il permet d'introduire une méthode à liaison dynamique (le choix de la méthode à exécuter, sera résolu durant l'exécution). En C++, on parle aussi de fonction membre virtuelle.
[ ] Il permet d'introduire une méthode à liaison statique (le choix de la méthode à exécuter, sera résolu à la compilation).

54. Quelle est la sortie du code C++ suivant ?

```
#include <iostream>
using namespace std;
void Somme(int a, int b, int & c)
{  a = b + c;
   b = a + c;
   c = a + b;
}
```

```
int main()
{  int a = 2, b =1;
   Somme(a, b, b);
   cout << a << " " << b;
   return 0;
}
```

[ ] 2 1 **[ ]** 2 3 **[ ]** 2 5 **[ ]** 3 5